

1. Таблица для сравнения способов хранения данных

критерии	Список смежности (Adjacency List)		Отдельная коллекция рёбер(Edge Collection)		Встраивание рёбер в узел(Embedded Edges)		Материализованные пути(Materialized Paths)	
	score	обоснование	score	обоснование	score	ообснование	score	обоснование
Скорость запроса соседей	5/5	Поиск по _id происходит мгновенно по индексу, Массив соседей извлекается за один запрос.	4/5	При наличии индексов по полям from и to запрос выполняется быстро. Немного медленнее, чем прямой доступ по массиву.	5/5	Все данные о вершине и её ребрах читаются за один запрос. Максимальная скорость чтения.	3/5	Пути уже предвычислены, но их нужно парсить, чем глубже обход, тем медленнее.
Сложность добавления связи	4/5	Добавление выполняется оператором \$push за один запрос. Не сложно, но нужно обновлять документ обеих вершин, если хранить двунаправленные связи	4/5	Добавление – это вставка нового документа в коллекцию routes. Немного сложнее, чем \$push (нужно создать отдельный документ).	5/5	Добавление выполняется оператором \$push во вложенный массив. Самый простой способ.	2/5	Очень сложно. При добавлении одного ребра нужно пересчитывать все пути для всех затронутых вершин.
Возможность хранения свойства рёбер	0/5	Невозможно. Массив хранит только ID соседей.	5/5	Полная поддержка. Можно хранить любые поля, что критически важно для алгоритма Дейкстры.	5/5	Полная поддержка. Вложенные объекты могут содержать любые поля.	5/5	Полная поддержка. В объектах путей можно хранить любые поля
ограничения	Средние	Массив ограничен 16 МБ. Нельзя фильтровать ребра по свойствам. Для поиска соседей второго уровня нужно несколько запросов или \$lookup	Средние	Для поиска всех соседей нужен запрос с \$or или два запроса. \$graphLookup медленный при глубине >3-4. Больше места в БД(каждое ребро – отдельный документ).	Серьёзные	Лимит 16 МБ на документ. Сложно обновлять конкретно одно ребро(нужно искать внутри массива). Сложно искать ребра,	критические	Огромное дублирование данных (одно и то же ребро может дублироваться в сотнях путей). При добавлении ребра нужно обновлять множество документов. Не

				Нет нативной оптимизации для графовых обходов.		входящие в вершину. Дублирование данных, если хранить ребра в обоих городах.		масштабируется для графов > 100 узлов.
Итогов ый score	9/15 + средние ограничения		13/15 + средние ограничения		15/15 + серьёзные ограничения		10/15 + критические ограничения	

Как устроены способы:

- 1. Список смежности – массив ID соседей : в документе вершины хранится массив с идентификаторами соседних вершин.
- 2. Отдельная коллекция рёбер(Edge Collection) : Вершины в одной коллекции, рёбра — в отдельной. Каждое ребро — документ с полями from и to.
- 3. Встраивание рёбер в узел: Рёбра хранятся как вложенные документы внутри вершины, с полной информацией о связи.
- 4. Материализованные пути : В документе вершины хранятся предвычисленные пути до других вершин.

Обоснование выбора подхода хранения графа:

Для реализации проекта выбран подход «Отдельная коллекция рёбер» (Edge Collection). Данный подход обеспечивает:

- 1. Хранение свойств рёбер — поля weight, airline, price могут быть добавлены в документы коллекции routes, что критически важно для реализации алгоритма Дейкстры.
- 2. Масштабируемость — отсутствие ограничения на количество рёбер у одной вершины позволяет моделировать граф европейских авиаперелётов любого размера без риска превысить лимит документа в 16 МБ.
- 3. Поддержку графовых запросов — оператор \$graphLookup позволяет эффективно выполнять рекурсивные запросы для поиска маршрутов с пересадками.
- 4. Гибкость — добавление, удаление и обновление рёбер выполняется стандартными операциями MongoDB без необходимости пересчёта путей (как в случае с материализованными путями).

Недостаток подхода (необходимость двух запросов для поиска всех соседей) компенсируется созданием составных индексов по полям from и to, а также использованием оператора \$or для объединения запросов.